



JSON В WPF

Вывод, добавление, редактирование, удаление

OBSERVABLECOLLECTION

Представляет динамическую коллекцию данных, которая предоставляет уведомления о добавлении или удалении элементов или при обновлении всего списка.

Большинство свойств и методов совпадают с обычным List.

```
1 using System.Collections.ObjectModel;  
2  
3 ObservableCollection<string> people = new ObservableCollection<string>();
```

Больше информации:

<https://metanit.com/sharp/tutorial/4.13.php>

<https://learn.microsoft.com/ru-ru/dotnet/api/system.collections.objectmodel.observablecollection-1?view=net-8.0>

OBSERVABLECOLLECTION

```
1 var people = new ObservableCollection<string>
2 {
3     "Tom", "Bob", "Sam"
4 };
5
6 // получаем первый элемент
7 Console.WriteLine(people[0]); // Tom
8 // изменяем первый элемент
9 people[0] = "Tomas";
10 Console.WriteLine(people[0]); // Tomas
```

```
1 using System.Collections.ObjectModel;
2
3 var people = new ObservableCollection<string>
4 {
5     "Tom", "Bob", "Sam"
6 };
7
8 foreach(var person in people)
9 {
10     Console.WriteLine(person);
11 }
12 for (int i = 0; i < people.Count; i++)
13 {
14     Console.WriteLine(people[i]);
15 }
```

OBSERVABLECOLLECTION

```
1 using System.Collections.ObjectModel;
2
3 var people = new ObservableCollection<string>();
4
5 // добавляем элемент
6 people.Add("Bob");
7 // вставляем элемент по индексу 0
8 people.Insert(0, "Tom");
9
10 // проверка наличия элемента
11 bool bobExists = people.Contains("Bob");    // true
12 Console.WriteLine($"Bob exists: {bobExists}");
13 bool mikeExists = people.Contains("Mike");  // false
14 Console.WriteLine($"Mike exists: {mikeExists}");
15
16 // удаляем элемент
17 people.Remove("Tom");
18 // удаляем элемент по индексу 0
19 people.RemoveAt(0);
```

ПРОСТОЕ ПРИЛОЖЕНИЕ

Вывод товаров,
добавление,
редактирование,
удаление.

Товары

Id: Название: Цена:

Id	Название	Цена
1	Ноутбук	30000
2	телефон	5555

Интерфейс

```
Title="Товары" Height="500" Width="800">
<Grid Margin="10">
  <Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*/>
  </Grid.RowDefinitions>

  <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
    <TextBlock Text="Id:" VerticalAlignment="Center" Margin="0,0,5,0"/>
    <TextBox x:Name="IdBox" Width="70" Margin="0,0,10,0"/>

    <TextBlock Text="Название:" VerticalAlignment="Center" Margin="0,0,5,0"/>
    <TextBox x:Name="NameBox" Width="180" Margin="0,0,10,0"/>

    <TextBlock Text="Цена:" VerticalAlignment="Center" Margin="0,0,5,0"/>
    <TextBox x:Name="PriceBox" Width="100" Margin="0,0,10,0"/>

    <Button Content="Добавить" Width="100" Margin="0,0,5,0" Click="Add_Click"/>
    <Button Content="Обновить" Width="100" Margin="0,0,5,0" Click="Update_Click"/>
    <Button Content="Сброс" Width="80" Click="Clear_Click"/>
  </StackPanel>

  <ListView x:Name="ProductsList"
    Grid.Row="1"
    SelectionChanged="ProductsList_SelectionChanged">
    <ListView.View>
      <GridView>
        <GridViewColumn Header="Id" Width="80" DisplayMemberBinding="{Binding Id}"/>
        <GridViewColumn Header="Название" Width="280" DisplayMemberBinding="{Binding Name}"/>
        <GridViewColumn Header="Цена" Width="120" DisplayMemberBinding="{Binding Price}"/>
      </GridView>
    </ListView.View>
  </ListView>
</Grid>
```

JSON

```
public class Product
{
    Ссылка: 8
    public int Id { get; set; }
    Ссылка: 4
    public string Name { get; set; } = "";
    Ссылка: 4
    public decimal Price { get; set; }
}
```

```
[
  {
    "Id": 1,
    "Name": "\u0414\u0435\u0443\u0442\u0431\u0443\u0443\u0430",
    "Price": 30000
  },
  {
    "Id": 2,
    "Name": "\u0442\u0435\u043b\u0435\u0444\u0445\u0435\u0434",
    "Price": 5555
  }
]
```

ЗАГРУЗКА ДАННЫХ ИЗ JSON В LISTVIEW

```
public partial class MainWindow : Window
{
    private const string FileName = "products.json";
    private ObservableCollection<Product> Products = new ObservableCollection<Product>();

    Ссылка: 0
    public MainWindow()
    {
        InitializeComponent();
        LoadProducts();
        // задаём коллекцию для содержимого ListView
        ProductsList.ItemsSource = Products;
    }

    Ссылка: 1
    private void LoadProducts()
    {
        // Если файл существует
        if (File.Exists(FileName))
        {
            // читаем всё содержимое файла
            string json = File.ReadAllText(FileName);
            // сериализуем содержимое в коллекцию
            Products = JsonSerializer.Deserialize
                <ObservableCollection<Product>>(json);
        }
    }
}
```

СОХРАНЕНИЕ ДАННЫХ В JSON

```
private void SaveProducts()
{
    string json = JsonSerializer.Serialize(Products, new JsonSerializerOptions()
    {
        WriteIndented = true
    });

    File.WriteAllText(FileName, json, Encoding.UTF8);
}
```

Ссылка: 3

```
private void ClearFields()
{
    IdBox.Clear();
    NameBox.Clear();
    PriceBox.Clear();
}
```

ДОБАВЛЕНИЕ ДАННЫХ В JSON

Ссылка 1

```
private void Add_Click(object sender, RoutedEventArgs e)
{
    Product product = new Product();

    product.Id = int.Parse(IdBox.Text);
    product.Name = NameBox.Text.Trim();
    product.Price = int.Parse(PriceBox.Text);

    // все элементы проходят условие,
    // т.е. нет элементов с Id нового продукта
    if (Products.Any(p => p.Id == product.Id))
    {
        MessageBox.Show("Товар с таким Id уже существует");
    }
    else
    {
        Products.Add(product);
        SaveProducts();
        ClearFields();
    }
}
```

ОБНОВЛЕНИЕ ДАННЫХ В JSON

Ссылка: 1

```
private void Update_Click(object sender, RoutedEventArgs e)
{
    if (!(ProductsList.SelectedItem is Product selected))
    {
        MessageBox.Show("Выберите товар для редактирования");
    }
    else if (int.Parse(IdBox.Text) == selected.Id)
    {
        Product update = Products.FirstOrDefault(p => p.Id == selected.Id);
        update.Name = NameBox.Text.Trim();
        update.Price = int.Parse(PriceBox.Text);

        ProductsList.Items.Refresh();
        SaveProducts();
        ClearFields();
    }
}
```

ВЫДЕЛЕНИЕ ЭЛЕМЕНТА И ОЧИЩЕНИЕ

Ссылка: 1

```
private void ProductsList_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (ProductsList.SelectedItem is Product p)
    {
        IdBox.Text = p.Id.ToString();
        NameBox.Text = p.Name;
        PriceBox.Text = p.Price.ToString();
    }
}
```

Ссылка: 1

```
private void Clear_Click(object sender, RoutedEventArgs e)
{
    ClearFields();
    ProductsList.SelectedItem = null;
}
```

УДАЛЕНИЕ ЭЛЕМЕНТА

```
<ListView x:Name="ProductsList"
    Grid.Row="1"
    SelectionChanged="ProductsList_SelectionChanged">
    <ListView.View>
        <GridView>
            <GridViewColumn Header="Id" Width="80" DisplayMemberBinding="{Binding Id}"/>
            <GridViewColumn Header="Название" Width="280" DisplayMemberBinding="{Binding Name}"/>
            <GridViewColumn Header="Цена" Width="120" DisplayMemberBinding="{Binding Price}"/>
            <GridViewColumn Header="Действие" Width="100">
                <GridViewColumn.CellTemplate>
                    <DataTemplate>
                        <Button Content="Удалить" Padding="8,3" Tag="{Binding}" Click="Delete_Click"/>
                    </DataTemplate>
                </GridViewColumn.CellTemplate>
            </GridViewColumn>
        </GridView>
    </ListView.View>
</ListView>
```

Tag="{Binding}" передаёт в кнопку весь объект строки, поэтому в Click можно сразу получить нужный Product.

```
private void Delete_Click(object sender, RoutedEventArgs e)
{
    Button btn = (Button)sender;
    if (btn.Tag is Product product)
    {
        Products.Remove(product);
        SaveProducts();
    }
}
```

ЗАДАНИЕ

- Реализуйте цифровую платформу по добавлению игр. В данном приложении реализуйте возможность добавление игры (Обложка, название, описание, рейтинг, цена (платно/бесплатно), возрастное ограничение, жанр). Далее пользователь может отредактировать данную информацию или уже удалить игру из платформы.
- Добавить проверку полей при добавлении/редактировании.
- Добавить предупреждения с вопросами да/нет на удаление и обновление с выводом ID, имени и цены.
- Добавить сообщение при сохранении данных в Json на обновление, добавление, удаление.